Лабораторная работа 12. Структуры данных: стеки

0. Создать обобщенный интерфейс для стека со следующими методами:

```
T Pop();              // извлечь элемент из вершины стека
void Push(T value);   // поместить элемент в стек
T Peek();             // показать элемент в вершине, не удаляя его из стека

bool IsEmpty();         // проверить, что стек пуст
bool IsFull();          // проверить что стек полон
```

1. Создать собственную реализацию стека на основе массива (элементы хранятся в обычном массиве ограниченной длины)**, этот класс должен реализовать интерфейс из п. 0.**
Добавить обработку исключений: при попытке Pop или Peek из пустого стека и при попытке добавления элементов в полный стек должны выбрасываться исключения с понятными сообщениями.

Протестировать стек на примере:
Из текстового файла прочитать все имеющиеся числа, разместить их в стеке в порядке чтения.
Вывести все содержимое стека на экран (числа должны выводиться в порядке обратном, тому, что были в файле, стек в конце операции должен стать пустым)

2. Использовать стек из п. 1 для задачи:
Из текстового файла читать строки, содержащие команды. Каждая строка содержит одну команду.
Команда — это либо "+ ", либо "−".
Команда "+ " означает добавление в стек числа.
Команда "−" означает изъятие элемента из стека.
Гарантируется, что цепочка команд не приведет к необходимости извлечением из пустого стека.
Гарантируется, что в процессе выполнения команд стек не будет переполнен.

Формат входного файла
В первой строке входного файла содержится число команд.
Каждая последующая строка исходного файла содержит ровно одну команду.
Формат вывода на экран
Вывести числа, которые удаляются из стека с помощью команды "−", по одному в каждой строке.
Числа нужно выводить в том порядке, в котором они были извлечены из стека.
Пример

input.txt	output
6 + 1 + 10 - + 2 + 1234 -	10 1234

3. Использовать стек из п. 1 для задачи:
Проверить правильность скобочной последовательности
Реализовать решение для скобок вида ( ), [ ], { }
*(*доп. – предложить удобный механизм добавления других видов скобок)*

В тестирующей программе из входного текстового файла читать строки, содержащие только ASCII-символы.
а) Выводить на экран

“YES” – если скобки в строке расставлены верно или NO, в противном случае

Пример

input.txt	output
()()	YES
([])	YES
([)]	NO
((]]	NO
)(	NO

б) выводить на экран

“YES” – если скобки в строке расставлены верно или порядковый номер символа в строке (номер скобки), если он создает ошибочную комбинацию

То есть вывести

- номер первой закрывающей скобки, для которой нет соответствующей открывающей.

Если такой нет, то

- то номер первой открывающей скобки, для которой нет соответствующей закрывающей.

Некоторые тестовые последовательности и результаты для них (последовательность -> результат):

```
([](){([])}) -> “YES”
{*} ->  “YES”
{} ->  “YES”
  ->  “YES”
*{} ->  “YES”
[] ->  “YES”
{}[] ->  “YES”
[()] ->  “YES”
(()) ->  “YES”
{[]}() ->  “YES”
([](){([])}) ->  “YES”
foo(bar); ->  “YES”
(slkj, {lk[lve]} ,l) -> “YES”
```

```
()[]}-> 5
{{[()]] ->  7
{{{[][][] ->  3
{*{{}) -> 3
[[*) -> 2
{{ ->  2
} ->  1
{{{**[][][] ->  3
() ( {} -> 3
[] ( [] -> 3
((({[]}) -> 2
{}([] -> 3
```

```
(slkj{lk[lsj]} -> 1
dasdsadsadas]]] -> 13
[]([]->  3
{{{[][][] ->  3
{ ->  1
{[} ->  3
()[]} ->  5
{{[()]] ->  7
foo(bar[i); ->  10
[]([] ->  3
```

4. Использовать стек из п. 1 для решения задачи.

Из текстового файла прочитать все имеющиеся числа, разместить их в стеке в порядке чтения. Выведите самое маленькое число и его порядковый номер в файле (относительно других чисел). Обеспечьте решение задачи за O(1).

Нужен будет стек с поддержкой минимума(максимума), достаточно просто реализуется двумя стековыми структурами.

5. Использовать стек из п. 1 для решения задачи.

а) преобразовать математическое выражение, заданное строкой, из инфиксной формы (обычной) в постфиксную (обратную польскую запись) с использованием стека. На выходе – строка с посфиксным выражением

b) реализовать калькулятор, который вычисляет значение выражения, заданного в постфиксной нотации